ЛАБОРАТОРНАЯ РАБОТА №2
SAST: поиск и исправление дефектов с Semgrep

Теория → пошаговая практика → проверка результата
2 академических часа · учебный проект vulnerable-app

Выполняем только на локальной учебной копии. Реальные секреты и чужие системы не используем.

1. Что вы изучите

Научиться запускать статический анализ, читать находки и исправлять причины SQL injection и XSS без необходимости быть опытным разработчиком.

• Запустите Semgrep и сохраните отчёт.

• Разберёте source и sink.

• Замените опасные SQL-строки на параметры.

• Уберите отключение экранирования.

• Проверите исправления тестами.

Опыт программирования не требуется: команды и фрагменты кода можно копировать. После каждого шага объясняется, что должно получиться и как проверить результат.

2. Теория простыми словами

Что делает SAST

SAST читает код без запуска приложения. Он ищет заданные шаблоны и пути данных. Например, request.form — вход пользователя, cursor.execute — выполнение SQL. Если строка SQL собирается из входа, правило сообщает об опасности.

Термины source, sink и sanitizer

Source — источник недоверенных данных. Sink — опасная операция. Sanitizer — безопасный механизм обработки. Для SQL основной защитой являются параметры: текст запроса и значения передаются отдельно.

Почему f-string в SQL опасен

f"...{username}..." вставляет пользовательский текст прямо в команду. Нарушитель может изменить её смысл. Параметр mark = '?' для SQLite или '%s' для PostgreSQL позволяет драйверу БД передать значение отдельно.

FP и FN

FP — ложное срабатывание: правило сообщило о проблеме, которой нет. FN — пропущенная проблема: например, наши правила не ищут IDOR. Поэтому нулевой отчёт не означает полную безопасность.

ГОСТ и документирование

ГОСТ Р 71207-2024 «Защита информации. Разработка безопасного программного обеспечения. Статический анализ программного обеспечения. Общие требования» требует организованного подхода к статическому анализу. В учебном отчёте сохраняем версию анализатора, конфигурацию, исходные данные, решение по находке и повторный результат.

Зачем нужны тесты после исправления

Сканер может перестать видеть шаблон, но приложение при этом сломается. Тесты подтверждают обычное поведение и невозможность повторить атаку. Исходные тесты намеренно подтверждают уязвимости — их нужно изменить.

3. Перед началом

• Распакуйте практикум и откройте терминал в каталоге vulnerable-app.

• Нужны Python 3.11+, браузер и терминал. Для ZAP и альтернативных запусков — Docker Desktop.

• Команды показаны для Git Bash. Для PowerShell используйте .venv\Scripts\Activate.ps1 вместо source.

• Создайте папку reports для результатов. Не публикуйте отчёты, пока не проверили их содержимое.

• Сохраняйте исходную копию проекта, чтобы сравнивать состояние до и после исправлений.

4. Практика по шагам

Шаг 1. Запустить Semgrep

Зачем: Сначала получим исходный отчёт. Ничего в коде пока не меняем.

Что сделать:

1. Откройте терминал в vulnerable-app.

2. Создайте папку reports.

3. Запустите локальные правила .semgrep.yml.

4. Если нативный Semgrep не работает на Windows, используйте Docker-команду ниже.

Команда или код:

```
mkdir -p reports
semgrep scan --config .semgrep.yml --metrics=off --json-output reports/sast-before.json .

# Альтернатива через Docker:
docker run --rm -v "${PWD}:/src" semgrep/semgrep:1.94.0 semgrep scan --config /src/.semgrep.yml
  --metrics=off --json-output /src/reports/sast-before.json /src
```

Ожидаемый результат: Semgrep находит 6 сообщений: две SQL injection, debug, токен и два safe. Ненулевой код при --error здесь означает находки, а не поломку анализатора.

Если не получилось:

• Ошибка semgrep-core.exe на Windows: используйте Docker.

• Нет reports: сначала выполните mkdir -p reports.

Шаг 2. Прочитать отчёт

Зачем: Важно понять находку, а не просто посчитать её.

Что сделать:

1. Найдите backend/app.py и две находки SQL.

2. Для каждой выпишите rule id, файл, строку и текст сообщения.

3. Найдите request.form или request.args — source.

4. Найдите cursor.execute — sink.

5. В таблице отметьте достоверность и влияние. Не удаляйте находки как шум.

Пример и объяснение: login: username приходит из формы → попадает в f-строку query → query передаётся cursor.execute. Это подтверждённый путь недоверенных данных.

Ожидаемый результат: Есть таблица минимум из шести строк. Для SQL указано, откуда приходят данные и где они попадают в запрос.

Шаг 3. Исправить SQL injection в /login

Зачем: Заменим ручную сборку SQL параметрами. Код можно скопировать, но важно понять каждую строку.

Что сделать:

1. Откройте backend/app.py и найдите функцию login.

2. Замените создание query и execute на пример ниже.

3. Оставьте остальную функцию без изменений.

4. Проверьте: mark — это параметр драйвера; username и password передаются кортежем отдельно.

Команда или код:

```
mark = database.placeholder()
user = database.get_db().cursor().execute(
    f"SELECT id, username, role FROM users WHERE username = {mark} AND password = {mark}",
    (username, password),
).fetchone()
```

Пример и объяснение: Было: SQL собирался как одна строка. Стало: {mark} обозначает место значения, а (username, password) передаёт два значения драйверу.

Ожидаемый результат: В login больше нет f-строки с SQL. Вход student/student123 продолжает работать.

Шаг 4. Исправить SQL injection в /search

Зачем: Применим тот же принцип к двум LIKE-условиям.

Что сделать:

1. Найдите функцию search в backend/app.py.

2. Сформируйте pattern со знаками % в Python, а не вставкой пользователя в SQL.

3. Используйте два mark и четыре параметра: pattern повторяется для title и content.

4. Проверьте поиск слова «Безопасная».

Команда или код:

```
pattern = f"%{query_text}%"
mark = database.placeholder()
cursor.execute(
    f"SELECT id, title, content FROM posts WHERE title LIKE {mark} OR content LIKE {mark}",
    (pattern, pattern),
)
```

Пример и объяснение: Если pattern = '%SQL%', драйвер ищет это как строку. Даже если внутри есть кавычка, она остаётся данными, а не частью команды.

Ожидаемый результат: Поиск работает, но запрос q больше не изменяет структуру SQL.

Шаг 5. Исправить XSS и debug

Зачем: Для HTML пользовательские значения должны отображаться как текст, а не как разметка.

Что сделать:

1. В frontend/templates/search.html замените {{ query|safe }} на {{ query }}.

2. В frontend/templates/comments.html замените {{ comment[2]|safe }} на {{ comment[2] }}.

3. В конце backend/app.py поставьте debug=False.

4. Добавьте в комментарий к исправлению объяснение, почему safe был опасен.

Команда или код:

```
{{ query }}
{{ comment[2] }}
app.run(host="0.0.0.0", port=5000, debug=False)
```

Ожидаемый результат: Jinja экранирует HTML. тест отображается текстом, а не становится жирным. Semgrep больше не показывает эти правила.

Шаг 6. Обновить тесты и повторить анализ

Зачем: Теперь тест должен подтверждать защиту, а не наличие уязвимости.

Что сделать:

1. В tests/test_app.py замените test_login_sql_injection_is_demonstrable на проверку отказа.

2. Замените тест хранимого HTML на проверку экранирования.

3. Запустите pytest и Semgrep повторно.

4. Сравните sast-before.json и sast-after.json.

Команда или код:

```
def test_login_sql_injection_is_blocked(client):
    response = client.post('/login', data={'username': "admin' -- ", 'password': 'wrong'})
    assert response.status_code == 200
    assert b'admin@example.test' not in response.data

def test_stored_content_is_escaped(client):
    client.post('/comments', data={'author': 'student', 'content': '<b>training-xss</b>'})
    response = client.get('/comments')
    assert '&lt;b&gt;training-xss&lt;/b&gt;' in response.get_data(as_text=True)

pytest -q
semgrep scan --config .semgrep.yml --metrics=off --json-output reports/sast-after.json .
```

Ожидаемый результат: pytest проходит. SQL-обход не авторизует пользователя, HTML экранирован, заданные находки Semgrep устранены. IDOR остаётся для ЛР5.

5. Что сдать в отчёте

• Версия Semgrep, команды и конфигурация.

• Таблица исходных находок с source/sink.

• Изменения в login, search, шаблонах и debug.

• Обновлённые тесты и их результат.

• Сравнение отчётов и перечень оставшихся рисков.

В конце добавьте краткий вывод: какой дефект был найден, какая причина устранена, что проверено повторно и какие риски остались.

6. Проверь себя

1. Почему параметры безопаснее конкатенации?

2. Что такое source и sink в /login?

3. Почему удаление safe меняет поведение HTML?

4. Чем FP отличается от FN?

5. Какие дефекты Semgrep в этой работе не проверяет?

7. Критерии оценивания

Критерий	Баллы
Запуск и отчёт	15
Разбор находок	20
Исправления SQL	25
Исправления XSS/debug	15
Тесты и сравнение	25

Максимум — 100 баллов. За использование чужой системы без разрешения или публикацию настоящих секретов работа не принимается. Честно описанный отрицательный результат и ограничения инструмента являются нормальной частью анализа.

8. Источники

• ГОСТ Р 56939-2024: https://www.gostinfo.ru/catalog/Details/?id=8094263

• ГОСТ Р 71207-2024: https://protect.gost.ru/gost/details/9dd6f064-6014-4416-a3ca-ef72ce0a6c3a

• OWASP ASVS: https://owasp.org/www-project-application-security-verification-standard/

• Semgrep CLI: https://semgrep.dev/docs/cli-reference